GET	/api/categories	Categories with their icon and banner image	200	-
GET	/api/organisations	The four charities and their contact details	200	-
GET	/api/weather/:eventId	Forecast for the event's venue, proxied from Open-Meteo	200	400, 404, 502
GET	/api/health	Reports the API version and whether the database answers	200	500

Worked example 1 - POST /api/registrations

Purpose. Creates a booking. This is the endpoint behind the Register button, and it is the only write endpoint the public site can reach.

Request body (JSON):

```
{
  "event_id": 5,
  "first_name": "Amelia",
  "last_name": "Ward",
  "email": "amelia.ward@example.com",
  "phone": "0412 345 678",
  "tickets": 2,
  "registration_date": "2026-10-20",
  "notes": "One vegetarian meal please"
}
```

Responses:

- **201 Created.** The row was inserted. The body returns the stored record including its new registration_id and the event's ticket price, and a Location header points at it.
- **400 Bad Request.** A field is missing or malformed - an empty first name, an email with no @, tickets of 0 or 51, or a date that is not a real calendar date. The body names the field and gives the reason, and the form shows that message under the offending input.
- **404 Not Found.** No event has that event_id.
- **409 Conflict.** Three different situations, each with its own message: this email has already booked this event; the event has been suspended; or the event has already taken place. Each one is a genuine conflict with the current state of the resource rather than a malformed request, which is exactly what 409 is for.

Worked example 2 - DELETE /api/events/:id

Purpose. Removes an event, but only when doing so destroys no bookings. The endpoint counts the registrations for the event first; if the count is not zero it returns 409 with the number of bookings in the message, so the admin screen can say 'this event has 4 bookings and cannot be deleted' rather than just failing. If the count is zero it deletes and returns 200. As noted in Section 4, the ON DELETE RESTRICT foreign key means the same rule is enforced even if a future version of this endpoint forgets to check.

Why these methods, and why these status codes

GET is used for every read, POST to create, PUT to replace an entire event, and DELETE to remove one. PUT rather than PATCH because the admin form always submits the whole event - every field is on the screen, pre-filled - so there is no such thing as a partial update in this application, and pretending otherwise would add a second code path for no benefit. The distinction that matters most is 400 versus 409: a 400 means 'your request was wrong, change it and try again', while a 409 means 'your request was fine, but the world is not in a state where it can be carried out'. A duplicate booking is the second kind; sending 400 for it would have told the visitor to fix something that was not broken.

Validation is not duplicated between the two write endpoints. A shared module, api/routes/validate.js, holds the field rules for an event and for a booking, so a rule can only be changed in one place and the public site and the staff site receive identical messages. Where the rule is also a database constraint, that is deliberate duplication of a different kind: the application check produces a good message, the constraint produces a guarantee.

Every query is parameterised. There is no string concatenation of user input anywhere in the codebase, LIMIT and OFFSET are coerced to bounded integers, the JSON body parser is capped at 100 kB, and CORS is an explicit allow-list of the two deployed origins rather than a wildcard. Credentials live in environment variables on the server and never reach the browser; the Angular environment files contain only the base URL of the API.

The weather endpoint is a proxy, on purpose

GET /api/weather/:eventId does not return the Open-Meteo response to the browser. It looks up the event's latitude and longitude, calls Open-Meteo from the server, translates the numeric weather codes into the plain descriptions the brief lists (0 -> clear sky, 61 -> light rain, and so on), and returns a compact object with the next few days. Three reasons: the browser never has to know that a third party exists, so the upstream URL can change without a rebuild; the mapping from code to words is testable JavaScript on the server rather than a lookup table repeated in a template; and when Open-Meteo is unreachable the API can answer 502 and the page says plainly that there is no forecast, instead of showing an empty box.

6. Client-side Website

Four pages, all Angular 22 components with their template, styles and TypeScript in separate files: Home, Search, Event details and Register. Each route carries a title, so the browser tab and the back button both say something meaningful.

Event details. The page calls GET /api/events/:id once and receives the event and its bookings together, so there is no second round trip and no chance of the two disagreeing. The 'Who is going' panel lists each booking with the name, the number of tickets and the date it was bought, in the order the API returned them - newest first, exactly as the brief asks. The Register button is a router link to /register?eventId=, so it works with the keyboard, with the back button and with a right-click 'open in new tab'.

Register. A Reactive Form with seven fields matching the table in Section 4. Validation runs as the visitor types and again on submit; each field shows its own message rather than one generic alert, and the submit button is disabled until the form is valid. The ticket quantity has a live total underneath it, computed from the event price, so the visitor sees what they are committing to before they commit. The booking date defaults to today and cannot be in the future. On success the form is replaced by a confirmation panel with the booking reference and a link back to the event, which is where the visitor naturally wants to go next.

Search. The filters from Assessment 2 (date, location, category, free text) are unchanged in behaviour but now run against the same endpoint the admin list uses, and the page keeps the chosen filters in the URL so a search result can be shared.

Weather panel. On the event page, a small panel shows the forecast for the venue over the next few days. It is a separate request from the event itself, so a slow or failing weather service never delays or breaks the page.

7. Admin-side Website

A separate Angular application, deployed under /admin/, with three screens and a persistent navigation bar. Authentication is out of scope per the brief, so the site opens straight onto the event list.

- **List.** Every event, whatever its status, with the name, date, venue, category, price, tickets sold, amount raised against the goal, and status. Filter chips switch between all, active, suspended and past, and a search box narrows by name. The status filter is a query parameter on GET /api/events rather than a filter applied in the browser, so the list stays correct however large the table grows.
- **New.** The same form component as the edit screen, empty. Required fields are marked and validated; the Save button calls POST /api/events and returns to the list with the new event highlighted.
- **Edit.** The form pre-filled from GET /api/events/:id, with that event's bookings listed underneath - name, email, tickets, date - and a total ticket count at the top, so staff can answer 'how full is this event?' without leaving the screen. Save calls PUT.
- **Delete.** A button on each row in the list. It asks for confirmation, then calls DELETE. If the API answers 409 the screen reports exactly why ('this event has 4 bookings and cannot be deleted') instead of showing a generic failure.

The navigation bar is part of the shell component, not of any page, so it is present on every screen by construction rather than by remembering to paste it into each template - which is also why a 404 route can redirect to the list without losing the menu.

8. Work Beyond the Minimum

The brief makes Angular and the weather API optional but rewards them, so this section records them separately from the required work.

Table 4 - Work that goes beyond the required feature list.

Extra	What it involved	Where to look
Angular 22 for the public site	Standalone components with no NgModules, signals and computed values instead of manual DOM wiring, HttpClient with firstValueFrom so every call returns a Promise, typed models for every API response, lazy route titles.	clientside/src/app/
Angular 22 for the staff site	A second application sharing the same conventions: Reactive Forms with custom validators, a shared form component reused by the new and edit screens, route parameters for :id, an interceptor-free service layer.	adminside/src/app/
Open-Meteo weather API	latitude/longitude added to the schema, a server-side proxy endpoint, weather codes mapped to text, graceful degradation when the upstream call fails.	api/routes/weather.js
Postman collection with assertions	Every endpoint in a runnable collection, including the 409 cases, so the API can be regression-tested in one click.	docs/postman/
One-click local start	start-all.bat runs the API and both Angular dev servers together, so the team and the marker do not have to remember three commands.	start-all.bat
Character-set hardening	Both the connection pool and the exported SQL file now pin utf8mb4, so the emoji icons survive a move to a host whose default is the three-byte character set.	api/event_db.js, database/

9. Deployment

Each team member deployed the whole stack to their own cPanel account, as the brief requires. The two deployments are identical in structure and differ only in the database name and the credentials.

Table 5 - The two live deployments.

Item	Jie Bao	Chengyu Gao
Public site	https://24831941.it.scu.edu.au/	https://24832054.it.scu.edu.au/
Admin site	https://24831941.it.scu.edu.au/admin/	https://24832054.it.scu.edu.au/admin/
API health	https://24831941.it.scu.edu.au/api/health	https://24832054.it.scu.edu.au/api/health
Database	jbao14_charityevents	cgao15_charityevents

The steps were: create the database and user in cPanel; import the exported SQL through phpMyAdmin; upload the API folder and register it as a Node.js application with server.js as the startup file, mounted at /api; run npm install; put the database credentials and the allowed origin into the application's environment; build each Angular app with ng build --base-href set for the directory it would live in and upload